

Real-time Neural MPC: Deep Learning Model Predictive Control for Quadrotors and Agile Robotic Platforms

Tim Salzmann¹, Elia Kaufmann², Jon Arrizabalaga¹, Marco Pavone³, Davide Scaramuzza² and Markus Ryll^{1,5}

Abstract—Model Predictive Control (MPC) has become a popular framework in embedded control for high-performance autonomous systems. However, to achieve good control performance using MPC, an accurate dynamics model is key. To maintain real-time operation, the dynamics models used on embedded systems have been limited to simple first-principle models, which substantially limits their representative power. In contrast to such simple models, machine learning approaches, specifically neural networks, have been shown to accurately model even complex dynamic effects, but their large computational complexity hindered combination with fast real-time iteration loops. With this work, we present *Real-time Neural MPC*, a framework to efficiently integrate large, complex neural network architectures as dynamics models within a model-predictive control pipeline. Our experiments, performed in simulation and the real world onboard a highly agile quadrotor platform, demonstrate the capabilities of the described system to run learned models with, previously infeasible, large modeling capacity using gradient-based online optimization MPC. Compared to prior implementations of neural networks in online optimization MPC we can leverage models of over 4000 times larger parametric capacity in a 50Hz real-time window on an embedded platform. Further, we show the feasibility of our framework on real-world problems by reducing the positional tracking error by up to 82% when compared to state-of-the-art MPC approaches without neural network dynamics.

Framework Code: <https://github.com/TUM-AAS/ml-casadi>
Experimental Code: <https://github.com/TUM-AAS/neural-mpc>

I. INTRODUCTION

MODEL Predictive Control (MPC) is one of the most popular frameworks in embedded control thanks to its ability to simultaneously address actuation constraints and performance objectives through optimization. Due to its predictive nature, the performance of MPC hinges on the availability of an accurate dynamics model of the underlying system. This requirement is exacerbated by strict real-time constraints, effectively limiting the choice of dynamics models on embedded platforms to simple first-principle models. Combining MPC with a more versatile and efficient dynamics model would

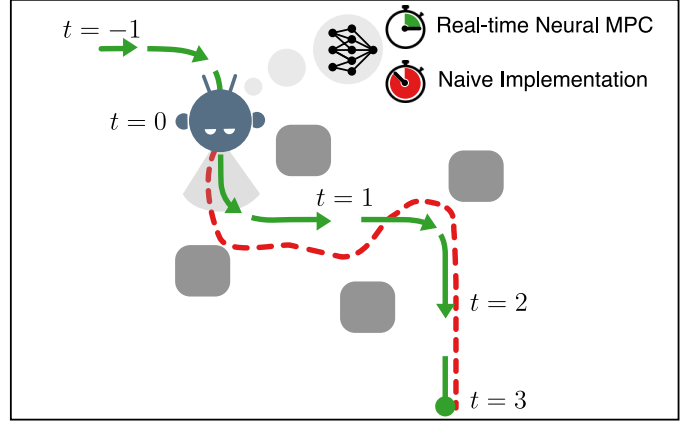


Fig. 1: Embedded Model Predictive Control using a neural network as learned dynamics model. Naive integration of the neural network in the MPC optimization loop would lead to extensive optimization times (red) resulting in instabilities. Our approach can handle complex larger learning models while being real-time capable (green).

allow for an improvement in performance, safety and operation closer to the robot's physical limits.

Precise dynamics modeling of autonomous systems is challenging, e.g. when the platform approaches high speeds and accelerations or when in contact with the environment. Accurate modeling is especially challenging for autonomous aerial systems, as high speeds and accelerations can lead to complex aerodynamic effects [1], and operating in close proximity to obstacles with an aerial vehicle requires modeling of interaction forces, e.g. ground effect. Data-driven approaches, in particular neural networks, demonstrated the capability to accurately model highly nonlinear dynamical effects [1], [2]. However, due to their large computational complexity, the integration of such models into embedded MPC pipelines remains challenging due to high frequency real-time requirements. To overcome this problem prior works have relied on one of two strategies:

(I) Largely reducing the model's capacity to the point where a lot of the predictive performance is lost but real-time speeds can be achieved [2]–[6]. Commonly, the model is reduced to a Gaussian Process (GP) with few supporting points [3], [4] or small neural networks [5]–[7]. Still, these methods are exclusively applied off-device on a powerful CPU.

(II) A control strategy different to online optimized MPC is used which are either non-predictive [8], [9], do not use online optimization [7], [10], [11], or learn the controller end-to-end [12]–[17].

Manuscript received: September, 2nd, 2022; Revised November, 27th, 2022; Accepted January, 24th, 2023.

This paper was recommended for publication by Editor Pauline Pounds upon evaluation of the Associate Editor and Reviewers' comments.

¹Tim Salzmann, Jon Arrizabalaga and Markus Ryll are with the Technical University of Munich {Tim.Salzmann, Jon.Arrizabalaga, Markus.Ryll}@tum.de

²Elia Kaufmann and Davide Scaramuzza are with the University of Zurich {ekaufmann, sdavide}@ifi.uzh.ch

³Marco Pavone is with the Stanford University and NVIDIA Research pavone@stanford.edu

⁵Munich Institute of Robotics and Machine Intelligence (MIRMI)

Digital Object Identifier (DOI): 10.1109/LRA.2023.3246839

In this paper, we present an efficient framework, *Real-time Neural MPC (RTN-MPC)*, that allows for the integration of large-capacity, data-driven dynamics models in online optimization Model Predictive Control and its deployment in real-time on embedded devices. Specifically, the framework enables the integration of arbitrary neural network architectures as dynamics constraints into the MPC formulation. To this end, *RTN-MPC* leverages CPU or GPU parallelized local approximations of the data-driven model. Compared to a naive integration of a deep network into an MPC framework, our approach allows unconstrained model architecture selection, embedded real-time capability for larger models, and GPU acceleration, without a decrease in performance.

Contribution

Our contribution is threefold: First, we formulate the computational paradigm for *RTN-MPC*, an MPC framework which uses deep learning models in the prediction step. By separating the computationally heavy data-driven model from the MPC optimization we can leverage efficient online approximations which allow for larger, more complex models while retaining real-time capability. Second, we compare and ablate the MPC problem with and without our *RTN-MPC* paradigm demonstrating improved real-time capability on CPU, which is further enhanced when GPU processing is available. Finally, we evaluate our approach on multiple simulation-based and real-world experiments using a high speed quadrotor in aggressive and close-to-obstacle maneuvers. All while running large models, multiple magnitudes higher in capacity compared to state-of-the-art algorithms, in a real-time window.

To the best of the authors' knowledge, this is the first approach enabling data-driven models, in a real-time **on-board** gradient-based MPC setting on agile platforms. Further, it scales to large models, vastly extending simple two or three-layer networks, on an off-board CPU or GPU enabling model sizes deemed unfit for closed-loop MPC [1], [2]. The introduced framework, while demonstrated on agile quadrotors, can be applied broadly and benefit any controlled agile system such as autonomous vehicles or robotic arms

II. RELATED WORK

With the advent of deep learning, there has been a considerable amount of research that aims to combine the representational capacity of deep neural networks with system modeling and control. In the following, we provide a brief overview of prior work that focuses on learning-based dynamics modeling, and data-driven control.

Data-driven Dynamics Models. Thanks to their ability to identify patterns in large amounts of data, deep neural networks represent a promising approach to model complex dynamics. Previous works that leverage the representational power of deep networks for such modeling tasks include aerodynamics modeling of quadrotors [1], [2], [18] and helicopters [19], turbulence prediction [20], tire friction modeling [21], and actuator modeling [22]. Although these works demonstrated that neural networks can learn system models that are able to learn the peculiarities of real-world robotic systems, they were restricted to simulation-only use cases or employed the network predictions as simple feedforward

TABLE I: Comparison of state-of-the-art data-driven MPC algorithms and their modeling capacity used for real-time (RT) applications. Prior works use models with small modeling capacity on high-end CPUs while our approach can leverage powerful models on an embedded platform.

	Model Architecture	RT Complexity	Parameters	RT Platform
DD-MPC [3]	Gaussian Process	20 Sup. Points	120	Intel i7
NNMPC [6]	MLP	2 Layer (64x64)	4096	Intel i7
KNODE-MPC [5]	MLP	1 Layer (32)	32	Intel i7
PI-TCN [2]	MLP	3 Layer(64x32x32)	3072	Laptop
Ours (<i>RTN-MPC</i>)	Diverse	Diverse	up to 11M	Jetson ARM/GPU

components in a traditional control pipeline: Saviolo *et al.* [2] had to revert their accurate physics-inspired model to a simple multi-layer-perceptron for closed-loop control.

Data-Driven Control. Leveraging the power of learned models in embedded control frameworks has been extensively researched in recent years. Most approaches have focused on combining the learned model with a simple reactive control scheme, such as the “Neural Lander” approach [8]. Neural Lander uses a learned model of the aerodynamic ground-effect to substantially improve a set-point controller in near-hover conditions. In [23], a learned recurrent dynamics model formulates a model-based control problem. While this approach allowed the system to adapt online to changing operating conditions, it cannot account for system constraints such as limited actuation input. Recent approaches that integrate the modeling strengths of data-driven approaches in the MPC framework propose the use of Gaussian Processes (GP) as a learned residual model for race cars [4] and quadrotors [3]. For Gaussian Processes, both their complexity and accuracy scale with the number of inducing points and with their dimensionality, limiting their performance on embedded systems. The approaches of Chee *et al.* [5], Williams *et al.* [7] and Spielberg *et al.* [6] follow the approach of [3] but model the quadrotor's dynamics residual using a small neural network for different applications. In Table I, we compare existing data-driven MPC approaches based on their modeling capacity. All state-of-the-art models are severely limited by the small modeling capacity of either GPs with a small number of supporting points or small two- or three-layer neural networks.

With the rise of deep reinforcement learning (RL), a new class of controllers for robotic systems has emerged that directly maps sensory observations to actions. Popular instances of such RL controllers are imitation learning of an expert controller [12], [13], [16] as well as Model-free and Model-based reinforcement learning [7], [10], [11], [17], [24]. Although such approaches achieve high control frequencies and may outperform online MPC approaches, they commonly require training in simulation, do not allow for tuning without costly retraining, and often discard the optimality, robustness and generalizability of an online optimized MPC framework.

Our work is inspired by [2]–[7] but replaces the Gaussian Process dynamics of [3], [4] or the small neural networks of [5], [6] with networks of higher modeling capacity [1], [2] and uses gradient-based optimization as opposed to a sampling-based scheme [7]. The resulting framework allows a combination of the versatile modeling capabilities of deep neural networks with state-of-the-art embedded optimization software without tightly constraining the choice of network architecture.

III. PROBLEM SETUP

In its most general form, MPC solves an optimal control problem (OCP) by finding an input command $\mathbf{u}$ which minimizes a cost function $\mathcal{L}$ subject to its system dynamics model $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ while accounting for constraints on input and state variables for current and future timesteps. Traditionally, the model f is manually derived from first principles using “simple” differential-algebraic equations (DAE) which often neglect complicated dynamics effects such as aerodynamics or friction as they are hard or computationally expensive to formalize. Following prior works [2]–[5], we partition f into a mathematical combination of first principle DAEs $f_{\mathcal{F}}$ and a learned data-driven model $f_{\mathcal{D}}$. This enables more general models extending the capability of DAE dynamics models. To solve the aforementioned OCP, we approximate it by discretizing the system into N steps of step size δt over a time horizon T using direct multiple shooting [25] which leads to the following nonlinear programming (NLP) problem

$$\begin{aligned} & \min_{\mathbf{u}} \sum_{k=0}^{N-1} \mathcal{L}(\mathbf{x}_k, \mathbf{u}_k) \\ \text{subject to} \quad & \mathbf{x}_{k=0} = \mathbf{x}_0 \\ & \mathbf{x}_{k+1} = \phi(\mathbf{x}_k, \mathbf{u}_k, f, \delta t) \\ & f(\mathbf{x}_k, \mathbf{u}_k) = f_{\mathcal{F}}(\mathbf{x}_k, \mathbf{u}_k) + f_{\mathcal{D}}(\mathbf{x}_k, \mathbf{u}_k) \\ & g(\mathbf{x}_k, \mathbf{u}_k) \leq 0 \end{aligned} \quad (1)$$

where $\mathbf{x}_0$ denotes the initial condition and g can incorporate (in-)equality constraints, such as bounds in state and input variables. ϕ is the numerical integration routine to discretize the dynamics equation where commonly a 4th order *Runge-Kutta* algorithm is used involving $E = 4$ evaluations of the dynamics function f . To leverage advancements in embedded solvers, the NLP is optimized using sequential quadratic programming (SQP) with ω being the SQP iterate $\omega^i = [\mathbf{x}_0^i, \mathbf{u}_0^i, \dots, \mathbf{x}_{N-1}^i, \mathbf{u}_{N-1}^i]$.

IV. BRINGING NEURAL MPC TO ONBOARD REAL-TIME

In this section, we lay down the key concepts to speed up the optimization times of MPC control with neural networks. The key insight in Section IV-A is that local approximations of the learned dynamics are sufficient to keep alike performance while drastically improving the generation process of the optimization problem. This insight is utilized in a three-phased embedded real-time optimization procedure in Section IV-B.

A. Locally Approximated Continuity Quadratic Program

Due to advances in embedded optimization solvers, SQP has become a well-suited framework to efficiently solve NLPs resulting from multiple shooting approximations of OCPs. This involves repetitively approximating and solving Eq. (1) as a quadratic program (QP). The solution to the QP leads to

an update on the iterate $\omega^{i+1} = \omega^i + \Delta\omega^i$ where the step $\Delta\omega^i$ is given by solving the following QP

$$\begin{aligned} & \min_{\Delta\omega^i} \sum_{k=0}^{N-1} \begin{bmatrix} \mathbf{q}_k \\ \mathbf{r}_k \end{bmatrix}^\top \begin{bmatrix} \Delta\mathbf{x}_k \\ \Delta\mathbf{u}_k \end{bmatrix} + \begin{bmatrix} \Delta\mathbf{x}_k \\ \Delta\mathbf{u}_k \end{bmatrix}^\top \mathbf{H}_k \begin{bmatrix} \Delta\mathbf{x}_k \\ \Delta\mathbf{u}_k \end{bmatrix} \\ \text{subject to} \quad & \Delta\mathbf{x}_{k+1} = \mathbf{A}_k \Delta\mathbf{x}_k + \mathbf{B}_k \Delta\mathbf{u}_k + \bar{\phi}_k - \mathbf{x}_{k+1}, \\ & k = 0, \dots, N-1, \\ & -\bar{\mathbf{g}}_k \geq \mathbf{G}_k^x \Delta\mathbf{x}_k + \mathbf{G}_k^u \Delta\mathbf{u}_k, \quad k = 0, \dots, N, \end{aligned} \quad (2)$$

where $\mathbf{q}_k = \frac{\delta}{\delta \mathbf{x}_k^i} \mathcal{L}(\mathbf{x}_k^i, \mathbf{u}_k^i)$, $\mathbf{r}_k = \frac{\delta}{\delta \mathbf{u}_k^i} \mathcal{L}(\mathbf{x}_k^i, \mathbf{u}_k^i)$ linearize the cost function and, under given circumstances, the hessian $\mathbf{H}_k$ can be approximated by the *Gauss-Newton* algorithm. $\bar{\phi}_k$ and $\bar{\mathbf{g}}_k$ are shorthand notations for the function evaluations $\phi(\mathbf{x}_k^i, \mathbf{u}_k^i, f, \delta t)$ and $g(\mathbf{x}_k^i, \mathbf{u}_k^i)$. The main computational burden lies in the parameter computation of the continuity condition Eq. (2). Specifically for each shooting node $k = 0, \dots, N-1$ we need to compute

$$\begin{aligned} \mathbf{A}_k &= \frac{\delta}{\delta \mathbf{x}_k^i} \phi(\mathbf{x}_k^i, \mathbf{u}_k^i, f, \delta t), \quad \mathbf{B}_k = \frac{\delta}{\delta \mathbf{u}_k^i} \phi(\mathbf{x}_k^i, \mathbf{u}_k^i, f, \delta t), \\ \bar{\phi}_k &= \phi(\mathbf{x}_k^i, \mathbf{u}_k^i, f, \delta t). \end{aligned}$$

Leading to $N * E * 2$ evaluations of the partial differentiations

$$\delta f(\mathbf{x}, \mathbf{u}) = \delta f_{\mathcal{N}}(\mathbf{x}, \mathbf{u}) + \delta f_{\mathcal{D}}(\mathbf{x}, \mathbf{u})$$

and $N * E$ function evaluations

$$f(\mathbf{x}, \mathbf{u}) = f_{\mathcal{N}}(\mathbf{x}, \mathbf{u}) + f_{\mathcal{D}}(\mathbf{x}, \mathbf{u})$$

of the dynamics equation. For computational heavy data-driven dynamics models $f_{\mathcal{D}}$ this leads to extensive processing times generating the QP.

The learned data-driven dynamics $f_{\mathcal{D}}$ are assumed to be accurate over the entire input space of states and controls present in the training dataset. However, to create the QP continuity condition we only require the model and its differentiations to be accurate in and around specific input values ω^i . Thus, to speed up the QP generation we replace the computationally heavy globally valid data-driven dynamics equation $f_{\mathcal{D}}$ with a computationally light locally valid approximation up to second order around the current iterate

$$\begin{aligned} f_{\mathcal{D}}^*(\mathbf{x}, \mathbf{u}) &\approx \bar{\mathbf{f}}_{\mathcal{D}}^i + \mathbf{J}_{\mathcal{D},k}^i \begin{bmatrix} \mathbf{x} - \mathbf{x}_k^i \\ \mathbf{u} - \mathbf{u}_k^i \end{bmatrix} \\ &+ \frac{1}{2} \begin{bmatrix} \mathbf{x} - \mathbf{x}_k^i \\ \mathbf{u} - \mathbf{u}_k^i \end{bmatrix}^\top \mathbf{H}_{\mathcal{D},k}^i \begin{bmatrix} \mathbf{x} - \mathbf{x}_k^i \\ \mathbf{u} - \mathbf{u}_k^i \end{bmatrix}. \end{aligned} \quad (4)$$

The required differentiations are readily available as submatrices of $\mathbf{J}_{\mathcal{D},k}^i$ for first-order approximations or as submatrices of a Tensor multiplication and sum for second-order approximations. The induced error of this computational simplification is of second order for a first-order approximation and of third order for a second-order approximation in the size of state and control changes between nodes. We will experimentally demonstrate this error to be neglectable for agile platforms where δt is small in Section VII.

Applying Eq. (4), the QP creation becomes independent of the complexity and architecture of the data-driven dynamics model. Further, with $\mathbf{J}_{\mathcal{D},k}^i$ and $\mathbf{H}_{\mathcal{D},k}^i$ being the single interfaces between the SQP optimization and the data-driven dynamics model, we are free to optimize the approximation

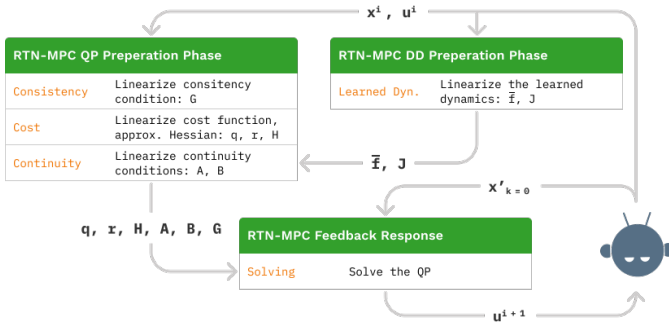


Fig. 2: Data flow for our *RTN-MPC* algorithm. The data-driven (DD) preparation phase is performed efficiently using optimized machine learning batch-differentiation tools on CPU or GPU.

process independent of the NLP framework; passing them as parameters to the continuity condition procedure of the QP generation. As f_D is a neural network model commonly consisting of large matrix multiplications we are therefore free to use algorithms and hardware optimized for neural network evaluation and differentiation. Those capabilities are readily available in modern machine learning tools such as PyTorch [26] and TensorFlow [27]. This enables us to calculate the Jacobians and Hessians for all shooting nodes N as a single parallelized batch on CPU or GPU.

B. Real-time Neural MPC

Even without a data-driven dynamics model, solving the SQP until convergence is computationally too costly in real-time for agile robotic platforms. To account for this shortcoming, MPC applications subjected to fast dynamics are commonly solved using a real-time-iteration scheme (RTI) [28], where only a single SQP iteration is executed - one quadratic problem is constructed and solved as a potentially sub-optimal but timely input command is preferred over an optimal late one. As shown in Fig. 2, *RTN-MPC* divides the real-time optimization procedure into three parts: *QP Preparation Phase*, *Data-Driven Dynamics Preparation Phase* and *Feedback Response*.

With available iterate ω^i , the data-driven dynamics preparation phase calculates $\tilde{f}_D^i$ and $\tilde{J}_{D,k}^i$ using efficient batched differentiates of the data-driven dynamics on CPU or GPU.

Meanwhile, the QP preparation phase constructs a QP by linearizing around x^i and control u^i using a first-order approximation $f_D^*(x, u)$ for the continuity condition parametrized by the result of the data-driven dynamics preparation phase.

Once a new disturbed state $x'_{k=0}$ is sensed, the feedback response phase solves the pre-constructed QP using the disturbed state as input. The iterate ω is adjusted with the QP result and the optimized command u is sent to the actuators.

C. Implementation

To demonstrate the applicability of the *RTN-MPC* paradigm, we provide a implementation using CasADi [29] and acados [30] as the optimization framework and PyTorch [26] as ML framework. This enables the research community to use arbitrary neural network models, trainable in PyTorch and usable in CasADi.

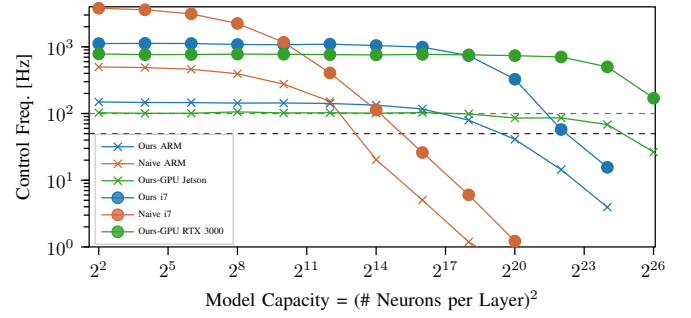


Fig. 3: Evaluation of real-time capability for different two-layer model parametric capacities. We evaluate on an embedded platform (Nvidia Jetson Xavier NX) and a laptop machine (Intel i7, Nvidia RTX 3000). Parametric model capacity is approximated by the squared number of neurons per layer. The *RTN-MPC* framework can run 4000 times larger models in parametric complexity compared to a naive implementation. To make the results comparable, we define a target run-time window of at least 50Hz (dashed red line) and preferably over 100Hz (dashed green line). However, in a real-world scenario the real-time window is specific to the use-case.

Further, we will compare our *RTN-MPC* approach against a naive implementation of a neural network data-driven MPC as applied in [2], [5], [6]. Here, the learned model is directly constructed in CasADi in the form of trained weight matrices and activation functions. Subsequently, the QP generation and automatic differentiation engine in CasADi has to deal with the full neural-network structure for which it is lacking optimized algorithms while being confined to the CPU.

V. RUNTIME ANALYSIS

We demonstrate the computational advantage of our proposed *RTN-MPC* paradigm compared to a naive implementation of a data-driven dynamics model in online MPC. Thus, we construct an experimental problem in which the nominal dynamics is trivial while the data-driven dynamics can be arbitrarily scaled in computational complexity. As such the nominal dynamics model is a double integrator on a position p while the data-driven dynamics is a neural network of variable architecture. To solely focus on the computational complexity of the data-driven dynamics, rather than modeling accuracy, the networks are not trained but weights are manually adjusted to force a zero output.

$$\dot{x} = \begin{bmatrix} \dot{p} \\ \ddot{p} \end{bmatrix} = f_{\mathcal{F}}(x, u) = \begin{bmatrix} \dot{p} \\ u \end{bmatrix}, \quad (5)$$

$$f(x, u) = f_{\mathcal{F}}(x, u) + \underbrace{f_D(x, u)}_0.$$

We use an explicit *Runge-Kutta* method of 4th order $\phi(x, u, f, \delta t) = RK4(x, u, f, \delta t)$ to numerically integrate f .

In this experiment, we simulate the system without any model-plant-mismatch to focus solely on runtime. The optimization problem is solved by constructing the multiple shooting scheme with $N = 10$ nodes.

Fig. 3 compares two-layer networks with increasing neuron count for a naive implementation and our *RTN-MPC* framework. On an embedded system, such as the Nvidia Jetson Xavier NX, our approach enables larger models of factor 60

TABLE II: Runtime comparison between naive implementation and *RTN-MPC*. Model complexity and parameter count are increasing from left to right. The naive approach becomes computationally costly in runtime above $\sim 10K$ parametric capacity with control frequencies dropping below 50Hz. Our approach can scale to powerful networks and complex network architectures showing real-time control frequencies of over 100Hz for over 50K parameters on an embedded device and over 13M on a GPU.

		Model Configuration									
		None	MLP								CNN
Architecture	Layers Neurons	-	2	2	5	5	12	12	20	50	18
		-	16	128	16	128	32	512	512	512	-
Parameter Count		0	354	17K	1.2K	67K	12K	182K	500K	13M	12M
		Control Freq. [Hz]									
Naive	ARM-CPU	-	403	20	280	6	66	2	<1	-	-
<i>RTN-MPC</i>	ARM-CPU	562	148	135	118	102	85	67	11	-	<1
	Jetson-GPU	-	109	107	88	84	63	61	46	-	9
Naive	i7-CPU	-	2228	116	1139	31	168	11	<1	<<1	-
<i>RTN-MPC</i>	i7-CPU	4262	1096	1071	885	784	588	507	91	39	4
	RTX3000-GPU	-	781	770	586	598	363	363	232	117	63

in parametric complexity on CPU and of factor 4000 on GPU while staying within a real-time window above 50Hz. Running on a desktop, which is the current default in data-driven MPC research [2], [3], [5], [21], we can run two-layer models with more than 150 million parameters above 100Hz on a low-end GPU (Nvidia RTX3000).

We further evaluate the runtime of a broad range of deep learning architectures in Table II. While the naive approach has better runtime for small networks, our approach dominates for larger and deeper networks enabling running a 12 layer 512 neurons each network above 50Hz on an embedded CPU and above 500Hz on a desktop CPU. To demonstrate that complex network architectures are easily integrated in the MPC loop using *RTN-MPC*, we run a full CNN ResNet model [31] with 18 convolutional layers in the optimization loop above 50Hz when leveraging the GPU capabilities of our framework.

VI. EXPERIMENTAL SETUP

While the *RTN-MPC* framework described in Section III can be applied to a variety of robotic applications, we will use agile quadrotor flight maneuvers to showcase its potential for real-world problems.

Notation. Scalars are denoted in lowercase s , vectors in lowercase bold $\mathbf{v}$, and matrices in uppercase bold $\mathbf{M}$. Coordinate frames such as the World W and Body B frames are defined with orthonormal basis i.e. $\{\mathbf{x}_W, \mathbf{y}_W, \mathbf{z}_W\}$, with the Body frame being located at the center of mass of the quadrotor (see Fig. 4). A vector from coordinate $\mathbf{p}_1$ to $\mathbf{p}_2$ expressed in the W frame is written as ${}_W\mathbf{v}_{12}$. If the vector's origin coincides with the frame it is described in, the frame index is dropped, e.g. the quadrotor position is denoted as $\mathbf{p}_{WB}$. Orientations are represented using unit quaternions $\mathbf{q} = (q_w, q_x, q_y, q_z)$ with $\|\mathbf{q}\| = 1$, such as the attitude state of the quadrotor body $\mathbf{q}_{WB}$. Finally, full SE3 transformations, such as changing the frame of reference from Body to World for a point $\mathbf{p}_{B1}$, are described by ${}_W\mathbf{p}_{B1} = {}_W\mathbf{t}_{WB} + \mathbf{q}_{WB} \odot \mathbf{p}_{B1}$. Note the quaternion-vector product denoted by $\odot$ representing a rotation of the vector $\mathbf{v}$ by the quaternion $\mathbf{q}$ as in $\mathbf{q} \odot \mathbf{v} = \mathbf{q}\mathbf{v}\bar{\mathbf{q}}$, where $\bar{\mathbf{q}}$ is the quaternion's conjugate.

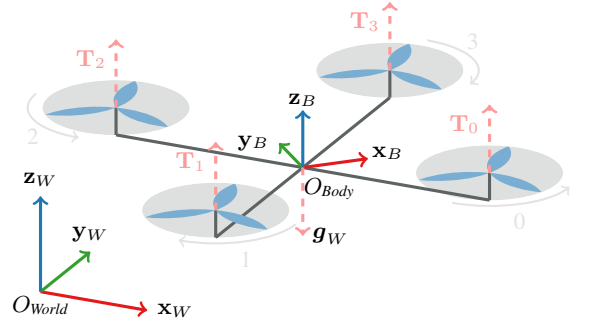


Fig. 4: Quadrotor model with world and body frames and propeller numbering convention. Grey arrows indicate the spinning direction of the individual rotors.

Nominal Quadrotor Dynamics Model. The nominal dynamics assume the quadrotor to be a 6 degree-of-freedom rigid body of mass m and diagonal moment of inertia matrix $\mathbf{J} = \text{diag}(J_x, J_y, J_z)$. Our model is similar to [3], [32], [33] as we write the nominal dynamics $\dot{\mathbf{x}}$ up to second order derivatives, leaving the quadrotors individual rotor thrusts $T_i \forall i \in (0, 3)$ as control inputs $\mathbf{u} \in \mathbb{R}^4$. The state space is thus 13-dimensional and its dynamics can be written as:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{WB} \\ \dot{\mathbf{q}}_{WB} \\ \dot{\mathbf{v}}_{WB} \\ \dot{\boldsymbol{\omega}}_B \end{bmatrix} = f_{\mathcal{F}}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{v}_W \\ \mathbf{q}_{WB} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_B/2 \end{bmatrix} \\ \frac{1}{m} \mathbf{q}_{WB} \odot \mathbf{T}_B + \mathbf{g}_W \\ \mathbf{J}^{-1} (\boldsymbol{\tau}_B - \boldsymbol{\omega}_B \times \mathbf{J} \boldsymbol{\omega}_B) \end{bmatrix} \quad (6)$$

with $\mathbf{g}_W = [0, 0, -9.81 \text{ m/s}^2]^\top$ denoting Earth's gravity, $\mathbf{T}_B$ the collective thrust and $\boldsymbol{\tau}_B$ the body torque. Again, an explicit *Runge-Kutta* integration of 4th order is used.

Augmented Aerodynamic Residual Models. Following previous works [3], [4], we use the data-driven model, in the form of a neural network $\mathcal{N}$, to complement the nominal dynamics by modeling a residual. In its full configuration, our residual dynamics model is defined as

$$\mathbf{f}(\mathbf{x}, \mathbf{u}) = f_{\mathcal{F}}(\mathbf{x}, \mathbf{u}) + f_{\mathcal{D}}(\mathbf{x}, \mathbf{u}), \quad f_{\mathcal{D}}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{0}_2 \\ f_{\mathcal{D}_\theta}(\mathbf{x}, \mathbf{u}) \\ f_{\mathcal{D}_\psi}(\mathbf{x}, \mathbf{u}) \end{bmatrix}, \quad (7)$$

where we individually account for disturbances in linear and angular accelerations unknown to the nominal dynamics and θ and ψ are the parameters of the neural networks modeling linear and angular disturbances respectively.

We also evaluate two simplified versions of the residual model:

$$\mathbf{f}_{\mathcal{D}_a}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{0}_2 \\ f_{\mathcal{D}_\theta}(\mathbf{v}_B) \end{bmatrix}, \quad \mathbf{f}_{\mathcal{D}_{a,u}}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{0}_2 \\ f_{\mathcal{D}_\theta}(\mathbf{v}_B, \mathbf{u}) \end{bmatrix}. \quad (8)$$

These simplified models only consider residual forces as a function of the platform's velocity (left), potentially accompanied by the commanded inputs (right).

Augmented Ground Effect Model. To show the strength of our approach, leveraging a complex arbitrary high level input, we extend the residual model using a height map under the quadrotor as additional input to model the ground effect.

$$\mathbf{f}_{\mathcal{D}_g}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{f}_{\mathcal{N}_\theta}(\mathbf{x}, \mathbf{u}, z_{WB} \cdot \frac{\mathbf{0}_2}{1} - h_l(\mathbf{p}_{WB}, \mathbf{H}_W)) \\ 0 \end{bmatrix}$$

where z_{WB} is the altitude of the quadrotor and h_l is a mapping $h_l : \mathbb{R}^3 \times \mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{3 \times 3}$ which takes the quadrotor's position $\mathbf{p}_{WB}$ and a fixed or sensed global height map $\mathbf{H}_W$ of size $N \times M$ as input. The function returns a 3×3 local patch of the height map around the quadrotor's position with a resolution of 10 cm.

MPC Cost Formulation. We specify the cost in Eq. (1) to be of quadratic form $\mathcal{L}(\mathbf{x}, \mathbf{u}) = \|\mathbf{x} - \mathbf{x}_r\|_Q^2 + \|\mathbf{u} - \mathbf{u}_r\|_R^2$ penalizing deviations from a reference trajectory $\mathbf{x}_r, \mathbf{u}_r$ and account for input limitations by constraining $0 \leq \mathbf{u} \leq \mathbf{u}_{\max}$.

VII. EXPERIMENTS

In our experiments we will re-validate the findings of previous works [2], [5] that using neural-network data-driven models in MPC improves tracking performance compared to no data-driven models or Gaussian Processes. More importantly, however, we will demonstrate that *RTN-MPC* enables the use of larger network capacities to fully exhaust possible performance gains while providing real-time capabilities.

All our experiments are divided into two phases: system identification and evaluation. During system identification, we collect data using the nominal dynamics model in the MPC controller. The state-control-timeseries are further processed in subsequent state, control tuples. Each step is then re-simulated using the nominal controller and the error is used as the training label for the residual model.

During evaluation we track two fixed evaluation trajectories, Circle and Lemniscate, and measure the performance based on the reference position tracking error. As such, we report the (Mean) Euclidean Distance between the reference trajectory and the tracked trajectory as error.

To identify model architectures used in the experiments we use a naming convention stating the model type followed by the size and the implementation type where we differentiate between our *RTN-MPC* approach (-Ours) and a naive integration (-Naive). N-3-32-Ours is a neural network model with 3 hidden layers, 32 neurons each using our *RTN-MPC* framework and N-3-32-Naive using a naive integration. GP-20 is a Gaussian Process Model with 20 inducing points.

All of our learned dynamic models are trained with a batch size of 64 and a learning rate of $1e^{-4}$ using the Adam optimizer. We split all datasets into a training and validation part and train the models using early stopping on the validation set. Dataset sizes are 20k datapoints for the simple simulation environment, 200k for the BEM simulation environment, and we use the openly available dataset presented in [1] with 1.8 million datapoints for the real-world experiment.

When comparing against GPs we follow the original implementation of [3] for the $\mathbf{f}_{\mathcal{D}_a}$ model configuration where one single-input-single-output GP is trained per dimension. For the $\mathbf{f}_{\mathcal{D}_{a,u}}$ configuration, their implementation is extended to a multi-input-single-output GP per dimension.

TABLE III: Results for the Simplified Simulation experiment. Our deep learning models outperform Gaussian Processes even using small models. For large models our *RTN-MPC* framework (-Ours) allows real-time capability without optimization time increase compared to the naive integration (-Naive).

Track	Error [mm]										t [ms]	
	Circle					Lemniscate					avg	avg
v_{avg} [m/s]	2	4.5	7	9.5	12	2	4.5	7	9.5	12		
v_{max} [m/s]	2.1	4.8	7.5	10.2	12.8	2.9	5.9	10.5	14.0	18.1		
Perfect	1	0	1	1	2	0	1	3	11	28	1.0	
Nominal	50	134	213	277	333	50	124	187	244	297	1.0	
GP-20	22	31	28	29	35	34	33	35	33	50	2.9	
GP-50	23	37	39	42	40	27	37	39	41	52	4.5	
GP-100	21	20	26	31	30	19	21	25	27	44	7.2	
N-1-12-Naive	33	33	35	34	33	28	32	33	32	48	1.6	
N-1-12-Ours	33	33	35	34	33	28	32	32	32	48	1.8	
N-2-18-Naive	21	22	27	31	30	19	23	27	30	43	2.1	
N-2-18-Ours	21	22	27	31	30	19	23	27	30	43	2.1	
N-3-32-Naive	13	17	22	27	26	14	19	26	28	45	8.2	
N-3-32-Ours	13	17	22	27	26	14	19	26	28	46	2.2	
N-4-64-Naive	10	14	18	23	23	11	17	27	27	45	35.9	
N-4-64-Ours	10	14	17	22	23	11	17	27	27	46	2.3	
N-5-128-Naive	13	18	22	28	29	16	19	31	29	47	178.7	
N-5-128-Ours	13	18	22	28	29	16	19	31	29	48	3.2	

A. Simulation

We use two simulation environments featuring varying modeling accuracy and real-time requirements to compare against a non-augmented MPC controller, a naive integration of data-driven dynamics [2], [5], [6], and GPs [3] with respect to real-time capability and model capacity.

Simplified Quadrotor Simulation. We use the simulation framework described in [3], where perfect odometry measurements and ideal tracking of the commanded single rotor thrusts are assumed. Drag effects by the rotors and fuselage are simulated, as well as zero mean ($\sigma = 0.005$) constant Gaussian noise on forces and torques, and zero mean Gaussian noise on motor voltage signals with standard deviation proportional to the input magnitude $\sigma = 0.02\sqrt{u}$. There are no run-time constraints as controller and simulator are run sequentially in simulated time. Using the simplified simulation, we analyze the predictive performance and run-time of our approach for varying network sizes and directly compare to the naive implementation and Gaussian Process approach. We constrain the residual model to linear accelerations $\mathbf{f}_{\mathcal{D}_a}$ to facilitate comparison with prior work [3]. To fairly evaluate the run-times of our full and distributed approach and considering the limited resources of embedded systems this experiment was performed on a single CPU core. The results are depicted in Table III. We also compare with a *Nominal* model where no learned residuals are modeled in the dynamics function and we also compare with an oracle-like *Perfect* model which uses the same dynamics equations as the simulation (excluding noise). Neural networks which achieve accurate modeling performance on the simulated dynamics are integrated easily with real-time optimization times below 3ms using our approach while they have high optimization times (up to 36ms) when a naive integration approach is used. The local approximations described in Section IV-A do not negatively influence performance compared to a naive implementation. Furthermore, we demonstrate that such modeling performance is not reachable with a GP even when using a large number of supporting points.

BEM Quadrotor Simulation. In addition to the simplified simulation setting, we also evaluate our approach in a highly accurate aerodynamics simulator based on Blade-Element-Momentum-Theory (BEM) [1]. In contrast to the simplified

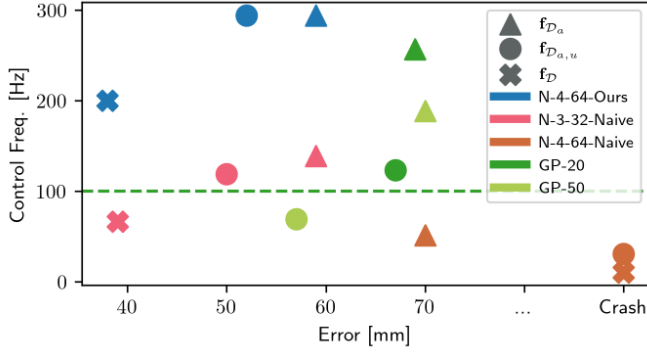


Fig. 5: Control Frequency over Tracking Error for Lemniscate trajectory in the realistic BEM simulation - top-left is desired. Our approach (blue) can leverage multidimensional inputs and large model capacities while being real-time capable. Increasing the naive approach to a four layer network (orange) leads to the controller becoming unstable for high-dimensional input. No additional noise is simulated, leading to error standard deviations within 1mm over 5 trials per experiment, induced by non-deterministic ROS transportation times.

TABLE IV: Results for the Real-World experiment. We improve tracking performance up to 82% compared to the nominal controller and up to 55% compared to GPs while being real-time capable unlike a naive integration. Result statistics are reported over 5 runs per experiment.

		Error [mm]	
Track		Circle	Lemniscate
$v_{\max}$ [m/s]		10	14
Eq. (6) $\mathcal{F}_{\mathcal{F}}$:	Nominal	321	359
Eq. (8) $\mathcal{f}_{D_a}$:	GP-20	66 ± 4	260 ± 11
Eq. (7) $\mathcal{f}_{\mathcal{D}}$:	N-3-32-Naive	crash	crash
Eq. (7) $\mathcal{f}_{\mathcal{D}}$:	N-3-32-Ours	59 ± 6	117 ± 9

simulation setting, this simulation can accurately model lift and drag produced by each rotor from the current ego-motion of the platform and the individual rotor speeds. The simulator runs in real-time and communicates with the controller via the Robot Operating System (ROS). We target a real-time control frequency of 100 Hz. We want to understand how our approach copes with increasing parameter count and model complexity of the learning task: First, we change the learned dynamics from just modeling linear acceleration residuals $\mathcal{f}_{D_a}$ with velocity as inputs to also accounting for rotor commands $\mathcal{f}_{D_{a,u}}$. In a second step we, model the full residual $\mathcal{f}_{\mathcal{D}}$ additionally outputting residuals on angular accelerations. The results obtained in each of these settings are illustrated in Fig. 5. While a naive approach can accurately model the residuals, its control frequency quickly declines for increasingly complex models. For larger networks, it has excessively high optimization times leading to the controller becoming unstable even in simulation. In contrast, our *RTN-MPC* approach can leverage both higher modeling capacity and the most representative residual model $\mathcal{f}_{\mathcal{D}}$ for on-par performance while running above 200Hz.

B. Real World

Finally, we perform experiments evaluating the real world effectiveness of our approach by performing a set of agile tra-

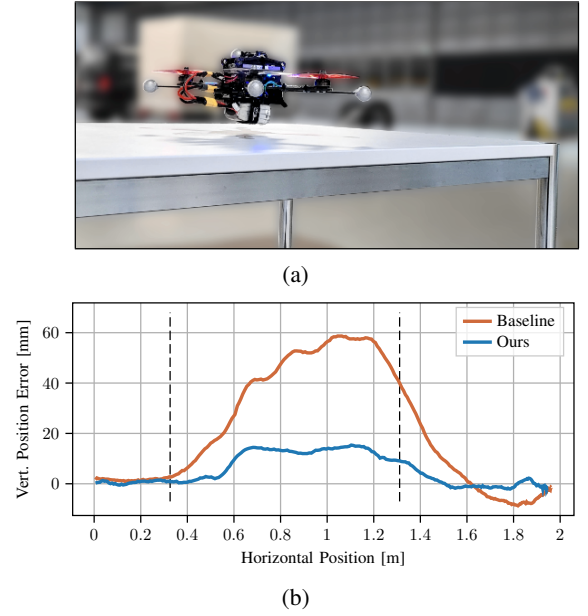


Fig. 6: (a) Quadrotor overflying the table in close proximity to the plane. (b) Vertical position error over distance. Vertical lines mark the position of the table. Our approach can model the aerodynamic effects in close proximity to the ground, substantially limiting the tracking error in z .

jectories using the physical quadrotor platform *agilicious* [34]. Control commands in the form of desired collective thrust and body rates are computed on a Jetson Xavier NX and are tracked by a low-level PID controller. All real-world flight experiments are performed in an instrumented tracking arena that provides accurate pose estimates at 400 Hz. As in the simulation experiments, we compare the tracking error along both circle and lemniscate trajectories at speeds up to 14 m s^{-1} . We evaluate our approach against the nominal controller, the naive integration, and the Gaussian Process configuration deployed in [3]. The results of these experiments is depicted in Table IV, where we improve positional tracking error by up to 82% compared to the nominal controller while the naive integration becomes unstable due to a long optimization time. Furthermore, we outperform Gaussian Processes by up to 55%.

Ground Effect.

Finally, we demonstrate the generalizability of our approach to other use-cases, modeling the complex aerodynamics of the ground effect using a height map as input (See Section VI). We place a table of 70 cm height in the flight arena and collect data by repeatedly flying over the table in close proximity. During evaluation, we fly repeated trajectories over the table with a target altitude of 80 cm of the quadrotor's center of gravity; leaving approximately 2 cm between the table and the lowest point of the quadrotor (battery). To isolate the performance of our approach, compensating for ground effect, we evaluate the trained model in two configurations. First, in which the height map information is unknown to the model (Baseline), and second where the information is known to the model. On an evaluation trajectory with 8 flyovers we improve the tracking error in z direction by 72% in close proximity (table plane +10 cm in xy) above of the table. A visualization of a single flyover can be seen in Fig. 6.

VIII. CONCLUSION

In this work we demonstrated an approach to scale the modeling capacity of data-driven MPC using neural networks to larger, more powerful architectures while being real-time capable on embedded devices. Our framework can improve new and existing applications of data-driven MPC by increasing the available real-time modeling capacity; making our approach generalizable to a variety of control applications.

An open challenge, which is not yet considered in this work, but the authors plan to tackle in the future, is to use a historic sequence of states and control input in a learned dynamics model. This would naturally lead to incorporating sequential and temporal models such (LSTMs, GRUs, and TCNs) in the optimization loop using our approach and would give rise to running approaches currently only feasible in simulation [1] in embedded MPC real-time.

We experimentally show that the controller's performance is not negatively affected by the real-time inducing approximations. Thus, this method overcomes the limitation of having to sacrifice performance for efficiency as described in previous works [2], [5]. We demonstrate its usefulness by evaluating the isolated real-time capability of *RTN-MPC* on different devices and applying the framework to the challenging problem of trajectory tracking of a highly agile quadrotor; reducing the tracking error substantially while using powerful models on-device.

ACKNOWLEDGMENT

We thank Matteo Zallio for his help in visually communicating our work.

REFERENCES

- [1] L. Bauersfeld, E. Kaufmann, P. Foehn, *et al.*, "NeuroBEM: Hybrid Aerodynamic Quadrotor Model," in *RSS*, 2021.
- [2] A. Saviolo, G. Li, and G. Loianno, "Physics-Inspired Temporal Learning of Quadrotor Dynamics for Accurate Model Predictive Trajectory Tracking," *IEEE RA-L*, 2022.
- [3] G. Torrente, E. Kaufmann, P. Foehn, *et al.*, "Data-Driven MPC for Quadrotors," in *IEEE RA-L*, 2021.
- [4] J. Kabzan, L. Hewing, A. Liniger, *et al.*, "Learning-Based Model Predictive Control for Autonomous Racing," *IEEE RA-L*, 2019.
- [5] K. Y. Chee, T. Z. Jiahao, and M. A. Hsieh, "KNODE-MPC: A Knowledge-Based Data-Driven Predictive Control Framework for Aerial Robots," *IEEE RA-L*, 2022.
- [6] N. A. Spielberg, M. Brown, and J. C. Gerdes, "Neural Network Model Predictive Motion Control Applied to Automated Driving With Unknown Friction," *IEEE Trans. on Control Systems Technology*, 2021.
- [7] G. Williams, N. Wagener, B. Goldfain, *et al.*, "Information theoretic MPC for model-based reinforcement learning," in *ICRA*, 2017.
- [8] G. Shi, X. Shi, M. O'Connell, *et al.*, "Neural Lander: Stable Drone Landing Control using Learned Dynamics," in *ICRA*, 2019.
- [9] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE RA-L*, 2017.
- [10] K. Chua, R. Calandra, R. McAllister, *et al.*, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models," in *NeurIPS*, 2018.
- [11] N. O. Lambert, D. S. Drew, J. Yaconelli, *et al.*, "Low-Level Control of a Quadrotor With Deep Model-Based Reinforcement Learning," *IEEE RA-L*, 2019.
- [12] E. Maddalena, C. da S. Moraes, G. Waltrich, *et al.*, "A Neural Network Architecture to Learn Explicit MPC Controllers from Data," *IFAC-PapersOnLine*, 2020.
- [13] J. Nubert, J. Köhler, V. Berenz, *et al.*, "Safe and Fast Tracking on a Robot Manipulator: Robust MPC and Neural Network Control," *IEEE RA-L*, 2020.
- [14] D. Wang, Z. J. Shen, X. Yin, *et al.*, "Model Predictive Control Using Artificial Neural Network for Power Converters," *IEEE Trans. on Industrial Electronics*, 2022.
- [15] R. Winqvist, A. Venkitaraman, and B. Wahlberg, "On Training and Evaluation of Neural Network Approaches for Model Predictive Control," in *arXiv preprint*, 2020.
- [16] E. Kaufmann, A. Loquercio, R. Ranftl, *et al.*, "Deep drone acrobatics," *RSS*, 2020.
- [17] M. Henaff, A. Canziani, and Y. LeCun, "Model-Predictive Policy Learning with Uncertainty Regularization for Driving in Dense Traffic," in *ICLR*, 2019.
- [18] S. Bansal, A. K. Akametalu, F. J. Jiang, *et al.*, "Learning quadrotor dynamics using neural network for flight control," in *IEEE Conf. on Decision and Control*, 2016.
- [19] A. Punjani and P. Abbeel, "Deep learning helicopter dynamics models," in *ICRA*, 2015.
- [20] Z. Li, N. B. Kovachki, K. Azizzadenesheli, *et al.*, "Fourier neural operator for parametric partial differential equations," in *ICLR*, 2020.
- [21] N. A. Spielberg, M. Brown, N. R. Kapania, *et al.*, "Neural network vehicle models for high-performance automated driving," *Science Robotics*, 2019.
- [22] J. Hwangbo, J. Lee, A. Dosovitskiy, *et al.*, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, 2019.
- [23] I. Lenz, R. Knepper, and A. Saxena, "DeepMPC: Learning Deep Latent Features for Model Predictive Control," in *RSS*, 2015.
- [24] O. M. Andrychowicz, B. Baker, M. Chociej, *et al.*, "Learning dexterous in-hand manipulation," *Int. Journal Robot. Research*, 2020.
- [25] H. Bock and K. Plitt, "A Multiple Shooting Algorithm for Direct Solution of Optimal Control Problems," *IFAC Proceedings Volumes*, 1984.
- [26] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019.
- [27] M. Abadi, A. Agarwal, P. Barham, *et al.*, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," 2016.
- [28] M. Diehl, H. Bock, J. P. Schlöder, *et al.*, "Real-time optimization and nonlinear model predictive control of processes governed by differential-algebraic equations," *Journal of Process Control*, 2002.
- [29] J. A. E. Andersson, J. Gillis, G. Horn, *et al.*, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 2019.
- [30] R. Verschuere, G. Frison, D. Kouzoupis, *et al.*, "acados: a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, 2021.
- [31] F. Ramzan, M. U. G. Khan, A. Rehmat, *et al.*, "A deep learning approach for automated diagnosis and multi-class classification of alzheimer's disease stages using resting-state fmri and residual neural networks," *Journal of medical systems*, 2020.
- [32] D. Falanga, P. Foehn, P. Lu, *et al.*, "PAMPC: Perception-Aware Model Predictive Control for Quadrotors," in *2018 IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2018.
- [33] M. Kamel, T. Stastny, K. Alexis, *et al.*, "Model Predictive Control for Trajectory Tracking of Unmanned Aerial Vehicles Using Robot Operating System," in *Robot Operating System (ROS)*, 2017.
- [34] P. Foehn, E. Kaufmann, A. Romero, *et al.*, "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight," *Science Robotics*, 2022.